Aircraft Damage Classification Using Fine-Tuning

Erwin S. De Jesus II
UP Training Center for Applied Geodesy and Photogrammetry, UP Diliman

Aircraft Damage Classification Using Fine-Tuning

I. Introduction

Aircraft safety relies on early detection of surface damage such as dents and cracks. Manual inspection is time-consuming and subjective which makes an automated system a necessity that can provide consistent, rapid assessments. This project develops a convolutional neural network (CNN) to classify images of aircraft surfaces into two categories: dent and crack. Using fine-tuning with a VGG16 backbone, the model learns discriminative features from a relatively small dataset.

II. Dataset

The dataset (aircraft_damage_dataset_v1) contains images organised into three subsets:

Split	Number of Images
Train	300
Valid	96
Test	50

Table 2.1 Splitting of dataset

Images are resized to 224×224 pixels (the input size expected by VGG16) and pixel values are normalised to [0,1]. Data is loaded using Keras ImageDataGenerator with binary labels.

III. Methodology

3.1 Fine-Tuning

A VGG16 model pre-trained on ImageNet is used as the feature extractor. The top classification layer is removed, and the remaining convolutional base is frozen to preserve learned features.

3.2 Custom Classifier Head

On top of the frozen base, we add:

- Flatten() layer
- Dense(512, activation='relu')
- Dropout(0.3)
- Dense(512, activation='relu')
- Dropout(0.3)
- Dense(1, activation='relu')

3.3 Training Configuration

- Loss: binary_crossentropy
- Optimizer: Adam with learning rate = 0.0001

- Metric: accuracy
- Epochs: 5
- Batch size: 32

IV. Results

4.1 Training History

Metric	Value
Training Loss	0.3204
Training Accuracy	0.8800
Validation Loss	0.5098
Validation Accuracy	0.7083

Table 4.1 Training History

After 5 epochs, the model achieved what is shown on Table 4.1

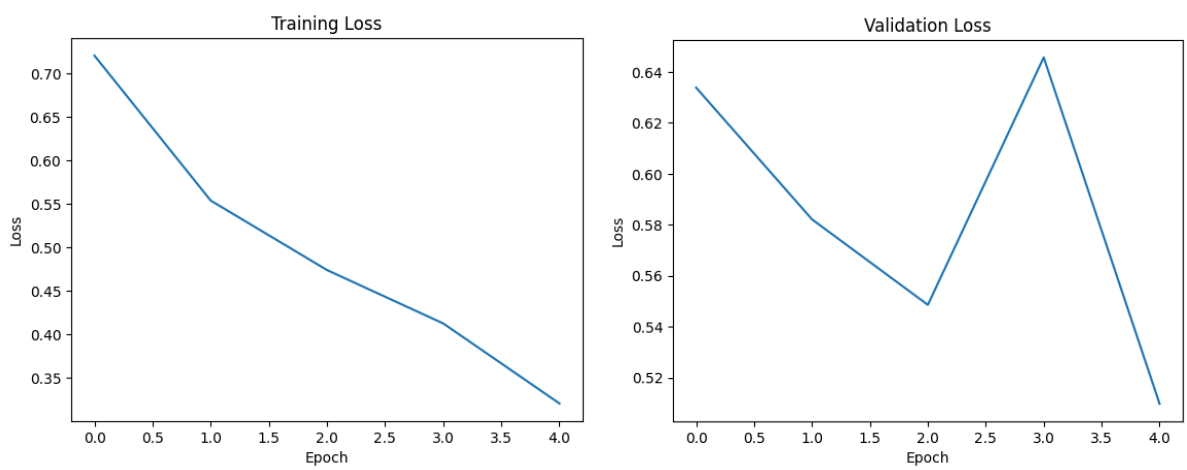

Figure 4.1 Loss Curves for Training and Validation

Figure 4.1 shows the plot of the loss during training and validation. This remarks that the loss gradually decreased linearly along with the number of epochs. For the validation, there was a spike in loss during the third epoch which suggests overfitting, however it reached its lowest loss during the fourth epoch and beyond.

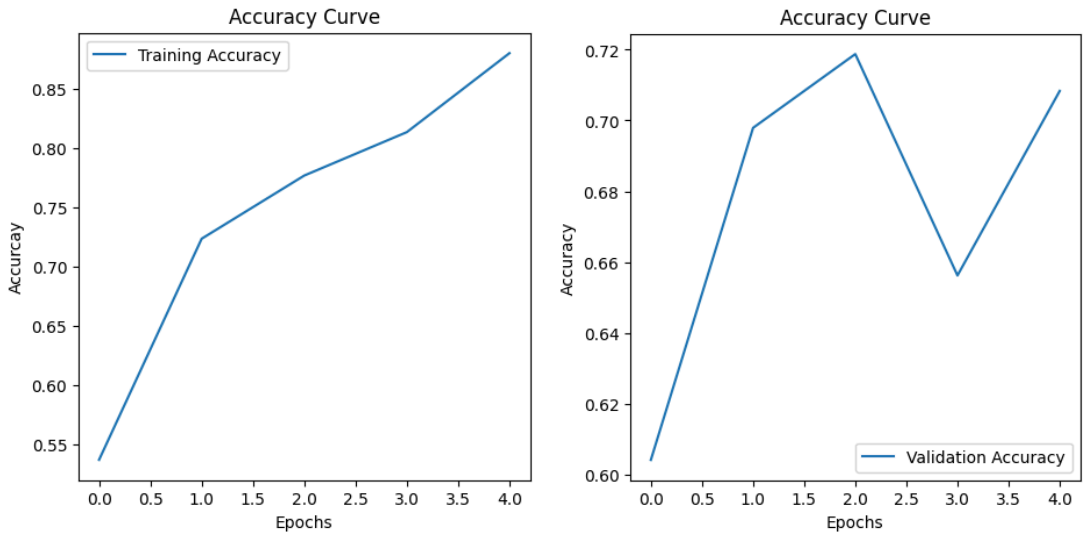

Figure 4.2 Accuracy Curves for Training and Validation

Figure 4.2 shows the plot of the accuracy during training and validation. This remarks that the accuracy gradually increased linearly along with the number of epochs. For the validation, there was a spike in accuracy during the third epoch which suggests overfitting, however it reached its lowest loss during the fourth epoch and beyond.

4.2 Test Performance

On the held-out test set (50 images):

- Test Loss: **0.7326**
- Test Accuracy: **0.6875**

4.3 Visual Inspection

Sample predictions show that the model correctly classifies many images, though misclassifications occur, especially on ambiguous damage patterns.

Figure 4.3 Sample Predictions

V. Discussion

Overfitting: The gap between training (88%) and validation/test (≈69%) accuracy indicates overfitting. This is expected given the small dataset; techniques such as data augmentation, additional dropout, or a simpler model could improve generalisation.

Fine-Tuning Value: Even with limited data, the VGG16 backbone provides rich features, enabling the model to reach a reasonable baseline in only 5 epochs.

Practical Implications: While not production-ready, the proof-of-concept demonstrates the feasibility of automating damage detection. With more data and fine-tuning, accuracy could be substantially improved.

VI. Conclusion

This project successfully implemented a binary image classifier for aircraft damage using transfer learning. The model distinguishes dents from cracks with ~69% test accuracy, highlighting both the power of CNNs and the challenges of small datasets. Future work will focus on expanding the dataset, applying data augmentation, and exploring more recent architectures (e.g., EfficientNet) to enhance performance.

Project Repository: https://github.com/esdeus/damage-detector-cnn.git
Technologies Used:

- Python
- TensorFlow/Keras

- VGG16
- OpenCV